

Teoría de la Computación, 2021-1

Nota de clase 4: Expresiones Regulares

Manuel Soto Romero

7 de abril de 2021
Colegio de Ciencia y Tecnología UACM

Continuaremos nuestro estudio de los Lenguajes Regulares por medio de un método basado en métodos algebraicos conocidos como *expresiones regulares*. Estas expresiones regulares (como veremos más adelante) pueden definir exactamente los mismos lenguajes que describen las gramáticas regulares y los autómatas finitos que estudiamos con anterioridad: lenguajes regulares.

Muchos sistemas como lenguajes de programación, procesadores de textos, sistemas gestores de bases de datos y muchos otros, incluyen este mecanismo como un método de procesamiento de cadenas. Como ejemplos de programas tenemos al comando **grep** que permite buscar texto en los sistemas operativos basados en UNIX o el software para construir analizadores léxicos JFLEX de gran importancia en el diseño e implementación de compiladores. En esta nota nos dedicaremos a estudiar este tipo de mecanismos.

4.1. Definición

Sea Σ un alfabeto. La *expresión regular* sobre Σ y los conjuntos que ellas denotan son definidos recursivamente como sigue:

- $\emptyset$ es una expresión regular y denota el conjunto vacío.
- ε es una expresión regular y denota el conjunto $\{\varepsilon\}$.
- Para cada a en Σ , a es una expresión regular y denota el conjunto $\{a\}$.
- Si r y s son expresiones regulares denotando el lenguaje R y S respectivamente entonces $(r + s)$, rs , r^* son expresiones regulares que denotan los conjuntos $R \cup S$, RS y R^* , respectivamente.

De esta forma, usamos a las expresiones regulares como una simplificación para denotar lenguajes. Recordando que los lenguajes son conjuntos podemos contrastar la definición anterior en la siguiente tabla:

Expresión Regular	Lenguaje
$\emptyset$	$\emptyset$
a	$\{a\}$
ε	$\{\varepsilon\}$
$(r + s)$	$R \cup S$
(rs)	RS
(r^*)	R^*

Los paréntesis sólo son agrupadores. Además, se tiene la siguiente prioridad y asociatividad de operadores:

- La cerradura $*$ asocia a la izquierda y tiene la mayor prioridad.
- Posteriormente se realiza la concatenación que asocia a la derecha.
- Finalmente se tiene el operador $+$ de alternativa que también asocia por la derecha.

Por ejemplo:

$$0 + 1b^* + a1^*$$

se resuelve en el siguiente orden:

$$(0 + ((1(b^*)) + (a(1^*))))$$

Veamos algunos ejemplos de expresiones regulares junto con el lenguaje que reconocen.

- ¿Qué lenguaje reconoce la siguiente expresión regular?

$$(0 + 1)^*$$

Analicemos cada subexpresión:

- 0 es la expresión regular que denota al lenguaje $\{0\}$
- 1 es la expresión regular que denota al lenguaje $\{1\}$
- Entonces $(0 + 1)$ es el lenguaje $\{0\} \cup \{1\} = \{0, 1\}$
- De esta forma $(0 + 1)^* = \{0, 1\}^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, 000, 001, \dots\}$

Es decir, reconoce el lenguaje de todas las posibles cadenas que se pueden formar como 0 y 1.

$$L_1 = \{w : w \in \Sigma^*\}$$

- ¿Qué lenguaje reconoce la siguiente expresión regular?

$$(a + b)c^*$$

Analicemos cada subexpresión:

- $a = \{a\}$
- $b = \{b\}$
- $c = \{c\}$
- $(a + b) = \{a, b\}$
- $c^* = \{\varepsilon, c, cc, ccc, cccc, \dots\}$
- $(a + b)c^* = \{a, b, ac, bc, acc, bcc, \dots\}$

El lenguaje admitido es el de todas las cadenas que comienzan con a o b seguidas de 0 o más símbolos c .

Observación 4.1. La representación de lenguajes regulares por medio de expresiones regulares no es única. Es posible que haya varias expresiones regulares para el mismo lenguaje. Por ejemplo, $b(a+b)^*$ y $b(b+a)^*$ representan el mismo lenguaje. Otro ejemplo sería con las expresiones $(a+b)^*$ y $(a^*b^*)^*$.

Ejercicio 4.1. Indica el lenguaje que reconocen las siguientes expresiones regulares:

1. $(a^*c) + (ab^*)$
2. $(00)^* + (1(11)^*)$

Veamos ahora, algunos ejemplos sobre diseño de expresiones regulares.

Ejemplo 4.1. Encontrar expresiones regulares que representen los siguientes lenguajes, definidos sobre el alfabeto $\Sigma = \{a, b\}$:

1. Lenguaje de todas las cadenas que comienzan con el símbolo b y terminan con el símbolo a .
 - Debemos garantizar que las cadenas inicien con b , lo cual puede ser representado por: b
 - Después del primer símbolo, puede haber cualquier número de as y bs , de forma tal que podemos generar dichas cadenas con la expresión: $(a+b)^*$.
 - Finalmente, la cadena debe terminar con a , con lo cual, la expresión regular más idónea es: a .
 - Concatenando las tres expresiones anteriores tenemos:

Solución: $b(a+b)^*a$

2. Lenguaje de todas las cadenas que tienen un número par de símbolos.
 - Dado que queremos pares de símbolos necesitamos construir cada uno de éstos, es decir, tanto el primer como el segundo símbolo pueden ser una a o una b y debemos concatenarlos. Esto se puede representar con $(a+b)(a+b)$.
 - Con la expresión del punto anterior, estamos construyendo cadenas de longitud 2, pero podemos tener otras longitudes, por ejemplo 0, 2, 4, 6, etc. Dado que ya concatenamos dos símbolos, esto se puede lograr aplicando la estrella de Kleene. Es decir:

Solución: $((a+b)(a+b))^*$

3. Lenguaje de todas las cadenas que tienen un número par de as .
 - Para reconocer cadenas que contengan un número par de as , partamos de un caso fácil, la cadena que tiene 2 de estos símbolos. En este caso, podemos tener alguna de las siguientes formas:
 - Dos símbolos al principio.
 - Dos símbolos al final.
 - Dos símbolos en medio.
 - Dos símbolos en cualquier posición.

Podríamos considerar todos estos casos y separarlos con una operación de suma, sin embargo, podemos auxiliarnos de la estrella de Kleene para indicar que puede haber cero o más presencias del símbolo b entre las dos a que queremos. Es decir: $b^*ab^*ab^*$.

- Ahora que tenemos fijado un par de as , podemos generalizarlo como hicimos en el ejemplo anterior con una estrella de Kleene. Es decir: $(b^*ab^*ab^*)^*$.
- Hasta el punto anterior, pareciera que ya tenemos la expresión regular, sin embargo, recordemos que el cero también es un número par, de forma que una cadena sin as también pertenece al lenguaje. Podemos agregar este paso con una suma, quedando:

Solución: $(b^*ab^*ab^*)^* + b^*$

Ejemplo 4.2. Encontrar expresiones regulares que representen los siguientes lenguajes, definidos sobre el alfabeto $\Sigma = \{0, 1\}$

1. Lenguaje de todas las cadenas que tienen exactamente dos ceros.

- Este es un caso particular de los ejemplos que vimos anteriormente. Dado que queremos exactamente dos ceros, simplemente colocamos los unos correspondientes con su respectiva estrella de Kleene entre los dos símbolos 0:

Solución: $1^*01^*01^*$

2. Lenguaje de todas las cadenas cuyo penúltimo símbolo, de izquierda a derecha es un 0.

- Queremos que el penúltimo símbolo sea un 0, de forma que antes de encontrar dicho símbolo, podemos tener cualquier número de ceros y unos, es decir: $(0 + 1)^*$
- Posteriormente, aparecerá el símbolo 0, que se reconoce con: 0
- Y finalmente, después de dichos patrones, podemos tener ya sea otro cero u otro uno, es decir: $(0 + 1)$
- Con lo cual, concatenando las 3 expresiones anteriores, tenemos:

Solución: $(0 + 1)^*0(0 + 1)$

Ejercicio 4.2. Sea $\Sigma = \{0, 1\}$. Encontrar una expresión regular que represente el lenguaje de todas las cadenas que no tienen dos ceros consecutivos.

Ejercicio 4.3. Sea $\Sigma = \{a, b, c\}$. Encontrar una expresión regular que represente el lenguaje de todas las cadenas que no contiene la cadena bc .

Referencias

- [1] Hopcroft, J. E; Montwani, R., and Ullman, J. *Introduction to Automata Theory Language, and Computation*. Addison Wesley, third edition 2007.
- [2] Rodrigo De Castro, *Notas de Clase de Teoría de la Computación: Lenguajes, autómatas, gramáticas*, Universidad Nacional de Colombia, Facultas de Ciencias, 2004.